

PaluNet

Documentation technique — Détection automatique du paludisme
Version 1.0.0 (cahier des charges V3.1)

Octobre 2026

Outil d'aide à la décision — ne remplace pas un diagnostic médical certifié.

Table des matières

1. Documentation technique PaluNet
2. Configuration
3. Pipeline de données (`src/data/`)
4. Modèle (`src/models/`)
5. API d'inférence (`src/api/`)
6. Interface web de démonstration (`web\_app/`)
7. Tests et qualité (`tests/`)
8. Déploiement et exploitation
9. Points d'attention, limites et pistes d'amélioration

Documentation technique PaluNet

Documentation de référence du code de PaluNet V1 (cahier des charges V3.1). Elle s'adresse aux développeurs, ML engineers et opérateurs qui doivent comprendre, modifier, entraîner, déployer ou auditer le système.

⚠ PaluNet est un **outil d'aide à la décision**. Il ne remplace pas un diagnostic médical certifié. La V1 est une version de démonstration : production gelée jusqu'à validation clinique.

1. Objectif du système

PaluNet classe l'image d'une **cellule sanguine unique** (frottis mince, coloration Giemsa) en deux classes :

- **Parasitized** : cellule infectée par *Plasmodium* (classe **positive**) ;
- **Uninfected** : cellule saine.

Le modèle est un **MobileNetV2** pré-entraîné sur ImageNet, adapté par transfer learning. Il est exporté en **ONNX** et servi par une **API FastAPI** sur CPU, sans TensorFlow. Une interface web montre le résultat et, en option, une carte **Grad-CAM** des zones qui ont motivé la décision.

La métrique prioritaire est le **rappel** de la classe positive. Un faux négatif (cellule infectée déclarée saine) coûte plus cher qu'un faux positif. Le seuil de décision est donc calibré pour garantir un **rappel** $\geq 98\%$ sur la validation, et n'est jamais fixé à 0,5 par défaut.

2. Arborescence

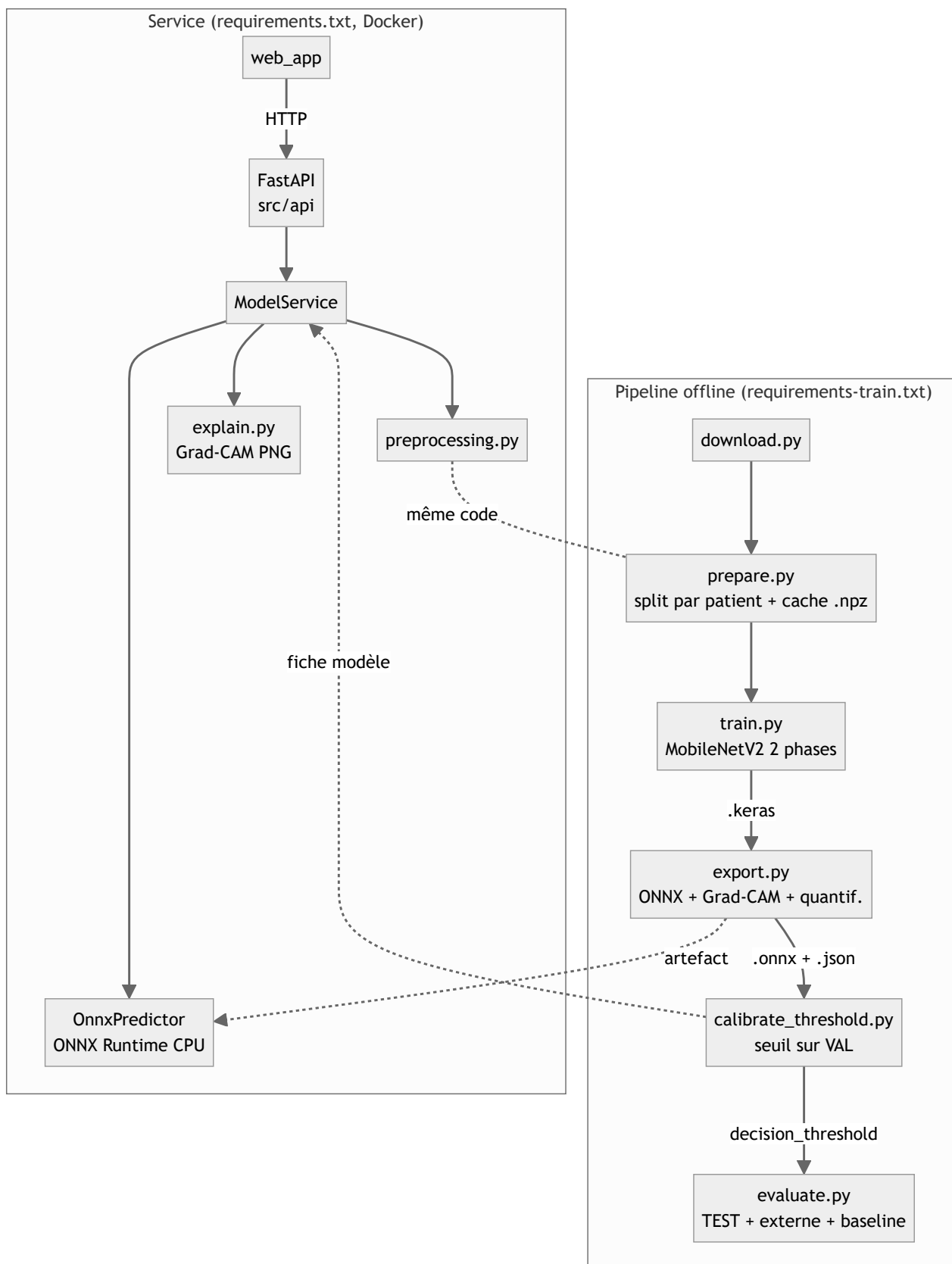
```
Palunet/
├── config/config.yaml          Source unique de vérité : hyperparamètres, class_map, version
├── src/
│   ├── config.py              Chargement de la config, encodage de classe (B9)
│   ├── utils/logger.py        Logs JSON, journal de prédictions anonymisé
│   ├── data/                  Pipeline de données (offline)
│   │   ├── download.py        A1 – récupération du dataset NIH
│   │   ├── split.py           A4 – split stratifié par patient
│   │   ├── preprocessing.py    A2 – prétraitement partagé entraînement / API
│   │   └── prepare.py         Construction des splits et du cache .npz
│   ├── models/               Modèle (offline + inférence)
│   │   ├── model.py           Architecture Keras + modèle d'inférence Grad-CAM
│   │   ├── train.py           B1-B4 – entraînement en 2 phases
│   │   ├── export.py          B5 – export ONNX + quantification
│   │   ├── calibrate_threshold.py B6 – calibration du seuil
│   │   ├── evaluate.py        B7/E4/E5 – évaluation, baseline, audit de dérive
│   │   ├── inference.py       Moteur ONNX Runtime, mock, règle de décision
│   │   ├── explain.py         Rendu de la carte Grad-CAM
│   │   ├── metrics.py         Métriques numpy pur (AUC, seuil...)
│   │   └── artifacts.py       Chemins et fiche modèle versionnés (B8)
│   └── api/                  API d'inférence (module C)
│       ├── main.py            Fabrique d'application, middleware, métriques
│       ├── settings.py        Configuration par variables d'environnement
│       ├── dependencies.py     Auth clé API, rate limiting, injection du service
│       ├── service.py          ModelService : chargement, validation, prédiction, logs
│       ├── schemas.py          Contrat Pydantic (figé, /v1)
│       └── routes/             health.py, predict.py
├── web_app/                  Interface de démonstration (index.html, app.js, styles.css)
├── tests/                    unit/, integration/, model/, load/
├── data/                     raw/, processed/, external/ (hors Git)
├── models/                   Artefacts ONNX + fiches JSON (hors Git)
├── metrics_baseline/         Baselines de non-régression (versionnées)
├── logs/                     Logs d'entraînement et de prédiction (hors Git)
├── Dockerfile, docker-compose.yml, .github/workflows/ci-cd.yml
└── requirements*.txt, pyproject.toml
```

3. Architecture

Le projet comporte deux mondes séparés par un artefact versionné :

1. **Pipeline offline** (TensorFlow) : données, entraînement, export, calibration, évaluation.
2. **Service en ligne** (sans TensorFlow) : API FastAPI + ONNX Runtime + interface web.

Le seul lien entre les deux est le couple `models/model_v<version>.onnx` + `models/model_v<version>.json` (fiche modèle). Le code de prétraitement `src/data/preprocessing.py` est partagé : il n'importe pas TensorFlow.



Dépendances entre modules

Module	Importé par
src/config.py	split, inference, settings, service, prepare, download, train, export, calibrate_threshold, evaluate
src/utils/logger.py	main, service, tous les scripts CLI
data/preprocessing.py	prepare, evaluate, service
data/split.py	prepare, evaluate
models/artifacts.py	settings, service, train, export, calibrate_threshold, evaluate
models/metrics.py	train, calibrate_threshold, evaluate
models/inference.py	service, export, calibrate_threshold, evaluate
models/explain.py	service
models/model.py	train, export
api/settings.py	service, dependencies, main
api/service.py	dependencies, routes/predict, main

Règle de conception : tout ce qui tourne dans l'API (config, preprocessing, inference, explain, metrics, artifacts, api/*) n'importe ni TensorFlow ni scikit-learn. L'image Docker reste ainsi légère.

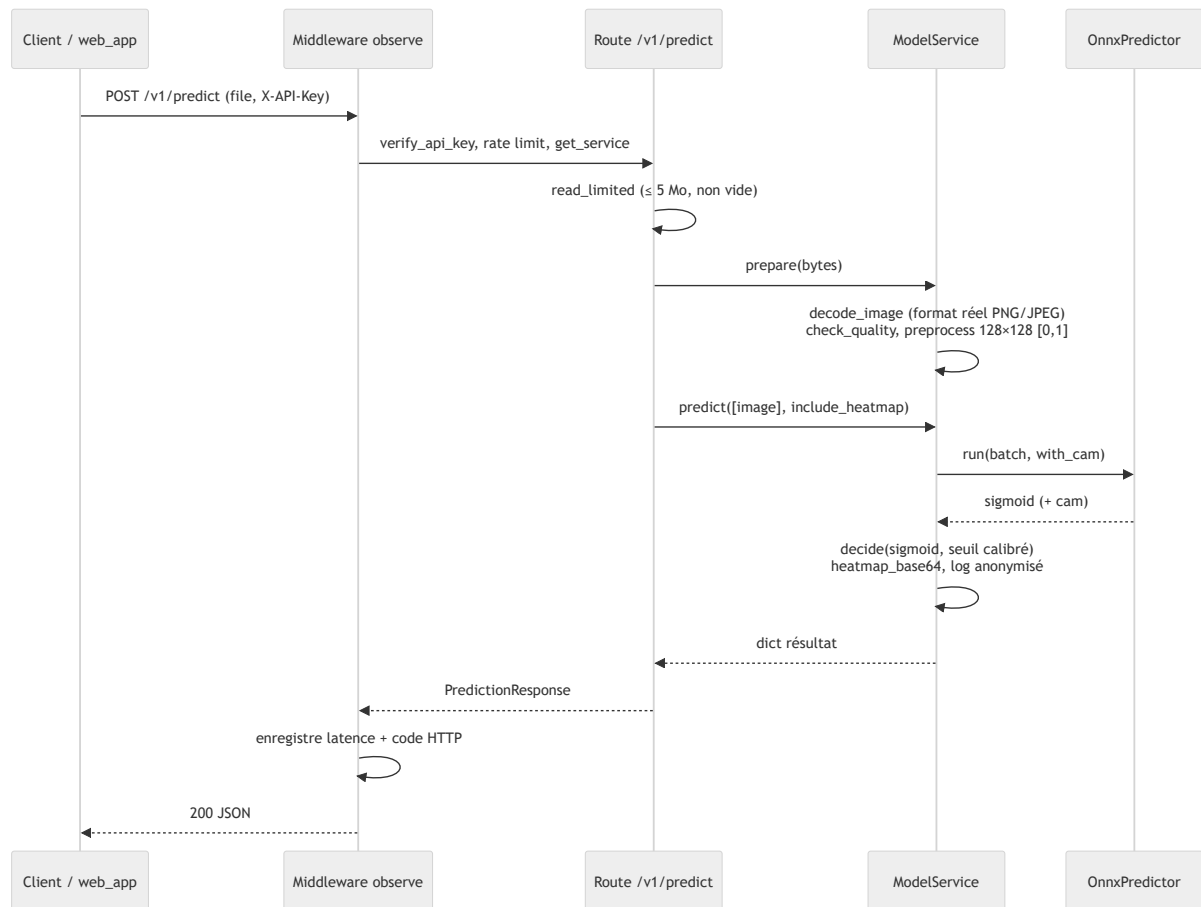
4. Flux de bout en bout

4.1 Entraînement (une fois par version de modèle)

```
python -m src.data.download --source "Malaria Cell Images Dataset.zip"
python -m src.data.prepare
python -m src.models.train
python -m src.models.export
python -m src.models.calibrate_threshold
python -m src.models.evaluate --write-baseline
```

Étape	Entrée	Sortie
download	URL NIH, archive ou dossier	data/raw/cell_images/{Parasitized,Uninfected}/*.png
prepare	images brutes	data/processed/{train,val,test,external}.npz, manifest.csv, split_summary.json
train	train.npz, val.npz	models/keras/model_v<v>.keras, logs/training_v<v>_phase{1,2}.csv, logs/experiments.csv
export	.keras, val.npz (vérification)	models/model_v<v>.onnx, models/model_v<v>.json (seuil = null)
calibrate_threshold	.onnx, val.npz	decision_threshold et calibration écrits dans la fiche JSON
evaluate	.onnx, fiche, test.npz, external.npz	métriques JSON, metrics_baseline/<v>.json

4.2 Prédiction (requête API)



5. Conventions transverses

- **Codes d'exigence.** Les docstrings citent les exigences du cahier des charges : A (données), B (modèle), C (API), D (interface), E (qualité / exploitation). Exemple : B6 = calibration du seuil. Voir `cdc-optimisé-détection-automatique-du-paludisme.md`.
- **Encodage de classe figé (B9).** `class_map = {0: Uninfected, 1: Parasitized}`. La sortie sigmoïde vaut $P(\text{classe d'indice } 1)$. Seule `positive_probability()` interprète cette orientation.
- **Chemins.** Tout chemin relatif est résolu depuis la racine du projet (`resolve_path`), pas depuis le répertoire courant.
- **Versionnement (B8).** Version semver `X.Y.Z` dans `config.yaml` ; tous les artefacts portent la version dans leur nom (`ModelPaths`).
- **Exécution.** Les scripts se lancent comme modules depuis la racine : `python -m src.<paquet>.<module>`.
- **Langue.** Code en anglais, docstrings, logs et messages d'erreur en français.

6. Glossaire

Terme	Définition
Rappel (recall, sensibilité)	$TP / (TP + FN)$: part des cellules parasitées détectées
Précision	$TP / (TP + FP)$: part des alertes qui sont de vrais positifs
Spécificité	$TN / (TN + FP)$: part des cellules saines reconnues

Terme	Définition
AUC	Aire sous la courbe ROC, indépendante du seuil
Seuil calibré	<code>decision_threshold</code> : probabilité minimale pour déclarer <code>Parasitized</code>
Fiche modèle	JSON à côté du <code>.onnx</code> : version, class_map, seuil, sha256, calibration
Baseline	<code>metrics_baseline/<version>.json</code> : métriques de référence pour la non-régression
Grad-CAM	Carte de chaleur des zones de l'image qui ont le plus pesé dans la décision
IQA	Image Quality Assessment : contrôle qualité minimal (taille, uniformité)
Mode mock	<code>MOCK_MODEL=true</code> : prédicteur factice pour développer sans modèle entraîné

Configuration

PaluNet a deux sources de configuration :

1. `config/config.yaml` : paramètres du **modèle et du pipeline** (hyperparamètres, encodage de classe, version). Lu par l'entraînement **et** par l'API.
2. **Variables d'environnement** : paramètres d'**exploitation** de l'API (clé, quota, chemin du modèle...). Lues par `src/api/settings.py`.

1. `config/config.yaml`

Référence des clés

Clé	Valeur par défaut	Utilisée par	Rôle
<code>seed</code>	42	split, train	Graine de reproductibilité (split par patient, sous-échantillonnage, init. Keras)
<code>class_map</code>	<code>{0: Uninfected, 1: Parasitized}</code>	partout	Encodage figé (B9) . La sigmoïde vaut $P(\text{class_map}[1])$
<code>positive_class</code>	<code>Parasitized</code>	partout	Classe « malade » : rappel prioritaire, <code>probability_parasitized</code>
<code>data.raw_dir</code>	<code>data/raw/cell_images</code>	download, prepare	Dossier contenant <code>Parasitized/</code> et <code>Uninfected/</code>
<code>data.processed_dir</code>	<code>data/processed</code>	prepare, train, export, calibrate, evaluate	Cache <code>.npz</code> et manifeste
<code>data.external_dir</code>	<code>data/external</code>	prepare	Jeu externe (A6), même structure que <code>raw_dir</code>
<code>data.nih_url</code>	URL NIH	download	Source officielle du dataset
<code>data.split.{train,val,test}</code>	<code>0.70 / 0.15 / 0.15</code>	prepare	Fractions du split par patient (somme = 1)
<code>preprocessing.image_size</code>	128	prepare, export, API	Côté de l'image carrée en entrée du modèle
<code>preprocessing.min_side_px</code>	32	API (IQA)	En dessous : avertissement « image trop petite »
<code>preprocessing.min_std</code>	<code>0.01</code>	API (IQA)	Écart-type des niveaux de gris en dessous duquel l'image est « quasi uniforme »
<code>augmentation.rotation_deg</code>	20	train	Rotation aléatoire $\pm 20^\circ$
<code>augmentation.translation</code>	<code>0.10</code>	train	Translation $\pm 10\%$ en hauteur et largeur
<code>augmentation.zoom</code>	<code>0.15</code>	train	Zoom $\pm 15\%$
<code>augmentation.horizontal_flip</code>	<code>true</code>	train	Retournement horizontal aléatoire

Clé	Valeur par défaut	Utilisée par	Rôle
<code>model.version</code>	"1.0.0"	partout	Version semver de l'artefact (B8)
<code>model.backbone</code>	<code>mobilenetv2</code>	(documentaire)	Non lu par le code : MobileNetV2 est codé en dur
<code>model.dense_units</code>	128	model	Neurones de la couche dense de la tête
<code>model.dropout</code>	0.3	model	Taux de dropout de la tête
<code>training.batch_size</code>	64	train	Taille de batch
<code>training.phase1</code>	epochs 15, lr 1e-4	train	Phase 1 : backbone gelé
<code>training.phase2</code>	epochs 10, lr 1e-5, unfreeze_last_layers 30	train	Phase 2 : fine-tuning des 30 dernières couches
<code>training.early_stopping</code>	monitor val_loss, patience 4	train	Arrêt anticipé avec restauration des meilleurs poids
<code>calibration.target_recall</code>	0.98	calibrate	Rappel minimal visé sur la validation (B6)
<code>export.opset</code>	13	export	Opset ONNX
<code>export.max_size_mb</code>	20	export, test modèle	Au-delà : quantification int8 dynamique
<code>acceptance.accuracy</code>	0.95	evaluate	Critère d'acceptation absolu (test)
<code>acceptance.recall</code>	0.96	evaluate	Idem
<code>acceptance.auc</code>	0.97	evaluate	Idem
<code>acceptance.tolerance</code>	0.005	evaluate, test modèle	Baisse tolérée par rapport à la baseline
<code>acceptance.drift_alert_recall</code>	0.94	evaluate -- images-dir	Seuil d'alerte de dérive (E5)

Changer la configuration

- Un changement d'hyperparamètre impose de relancer le pipeline d'entraînement complet.
- Un nouveau modèle doit recevoir une **nouvelle version** (`model.version`), sinon les artefacts de la version précédente sont écrasés. Voir plan-derive.md pour la convention semver.
- **Ne pas inverser** `class_map` sur un modèle existant : l'API refuse de démarrer si la fiche modèle ne correspond pas (contrôle B9). Le code reste cohérent si on inverse puis réentraîne, car toute l'interprétation passe par `positive_probability()`.

2. Module `src/config.py`

Élément	Description
<code>ROOT_DIR</code>	Racine du projet (parent de <code>src/</code>). Base de tous les chemins relatifs
<code>DEFAULT_CONFIG_PATH</code>	<code>ROOT_DIR / "config" / "config.yaml"</code>
<code>load_config(path=None)</code>	Lit le YAML (chemin explicite, sinon <code>\$CONFIG_PATH</code> , sinon défaut). Normalise et valide <code>class_map</code> . Résultat mis en cache (<code>lru_cache</code> , 4 entrées) : le dict retourné est partagé, ne pas le modifier

Élément	Description
<code>parse_class_map(raw)</code>	Convertit les clés en <code>int</code> et les valeurs en <code>str</code> (YAML et JSON peuvent donner des clés <code>str</code>)
<code>validate_class_map(class_map, positive_class)</code>	Lève <code>ValueError</code> si les clés ne sont pas exactement <code>{0, 1}</code> , si les noms sont dupliqués, ou si <code>positive_class</code> est absente
<code>label_of(class_name, class_map)</code>	Nom de classe → indice. <code>KeyError</code> si inconnu
<code>positive_probability(sigmoid_out, class_map, positive_class)</code>	Sortie sigmoïde → probabilité de la classe positive. Renvoie <code>sigmoid_out</code> si la classe positive a l'indice 1, sinon <code>1 - sigmoid_out</code> . Fonctionne sur un scalaire ou un tableau numpy. Seul endroit où l'orientation de la sortie est interprétée
<code>resolve_path(p)</code>	Chemin absolu inchangé ; chemin relatif → <code>ROOT_DIR / p</code>

Exemple :

```
from src.config import load_config, positive_probability
cfg = load_config()
p = positive_probability(0.93, cfg["class_map"], cfg["positive_class"]) # 0.93 = P(Parasitized)
```

3. Variables d'environnement de l'API

Lues une seule fois au démarrage par `Settings.from_env()`.

Variable	Défaut	Type	Rôle
<code>CONFIG_PATH</code>	<code>config/config.yaml</code>	chemin	Autre fichier de configuration (lu par <code>load_config</code>)
<code>MODEL_PATH</code>	<code>models/model_v<model.version>.onnx</code>	chemin	Artefact ONNX. La fiche <code>.json</code> de même nom doit être à côté
<code>MODEL_VERSION</code>	fiche modèle	str	Surcharge la version affichée
<code>DECISION_THRESHOLD</code>	fiche modèle	float]0,1[	Surcharge le seuil calibré. Vide = ignoré
<code>MOCK_MODEL</code>	<code>false</code>	bool	Prédicteur factice (développement uniquement). Seuil 0,5 non calibré
<code>API_KEY</code>	vide	liste CSV	Clés acceptées dans <code>X-API-Key</code> . Vide = authentification désactivée
<code>RATE_LIMIT</code>	<code>60/minute</code>	syntaxe <code>limits</code>	Quota par clé (par IP si pas de clé) sur chaque route <code>/v1/*</code>
<code>RATE_LIMIT_STORAGE</code>	<code>memory://</code>	URI	Stockage du quota. <code>redis://...</code> pour le partager entre instances
<code>MAX_UPLOAD_MB</code>	5	float	Taille maximale d'un fichier (→ 413)
<code>MAX_BATCH_SIZE</code>	32	int	Nombre maximal de fichiers par lot (→ 400)
<code>LOG_LEVEL</code>	<code>INFO</code>	str	Niveau du logger racine
<code>PREDICTION_LOG_FILE</code>	vide	chemin	Fichier du journal de prédictions (rotation quotidienne UTC)

Variable	Défaut	Type	Rôle
LOG_RETENTION_DAYS	365	int	Nombre de fichiers journaliers conservés
INTRA_OP_THREADS	auto	int	Threads ONNX Runtime. Aligner sur les vCPU du conteneur
CORS_ORIGINS	vide	liste CSV	Origines autorisées. Vide = pas de middleware CORS
SERVE_WEB_APP	true	bool	Monte web_app/ sur /
PORT	8000	int	Utilisé par le CMD Docker et le healthcheck uniquement

Booléens : 1, true, yes, on (insensible à la casse) valent vrai ; toute autre valeur vaut faux.

Priorité des sources pour le modèle servi

Paramètre	1 ^{re} source	2 ^e source	Absent
Version	MODEL_VERSION	model_version de la fiche	None
Seuil	DECISION_THRESHOLD	decision_threshold de la fiche	Erreur : API non prête (503)
class_map	—	fiche, comparée à config.yaml	Erreur si différente

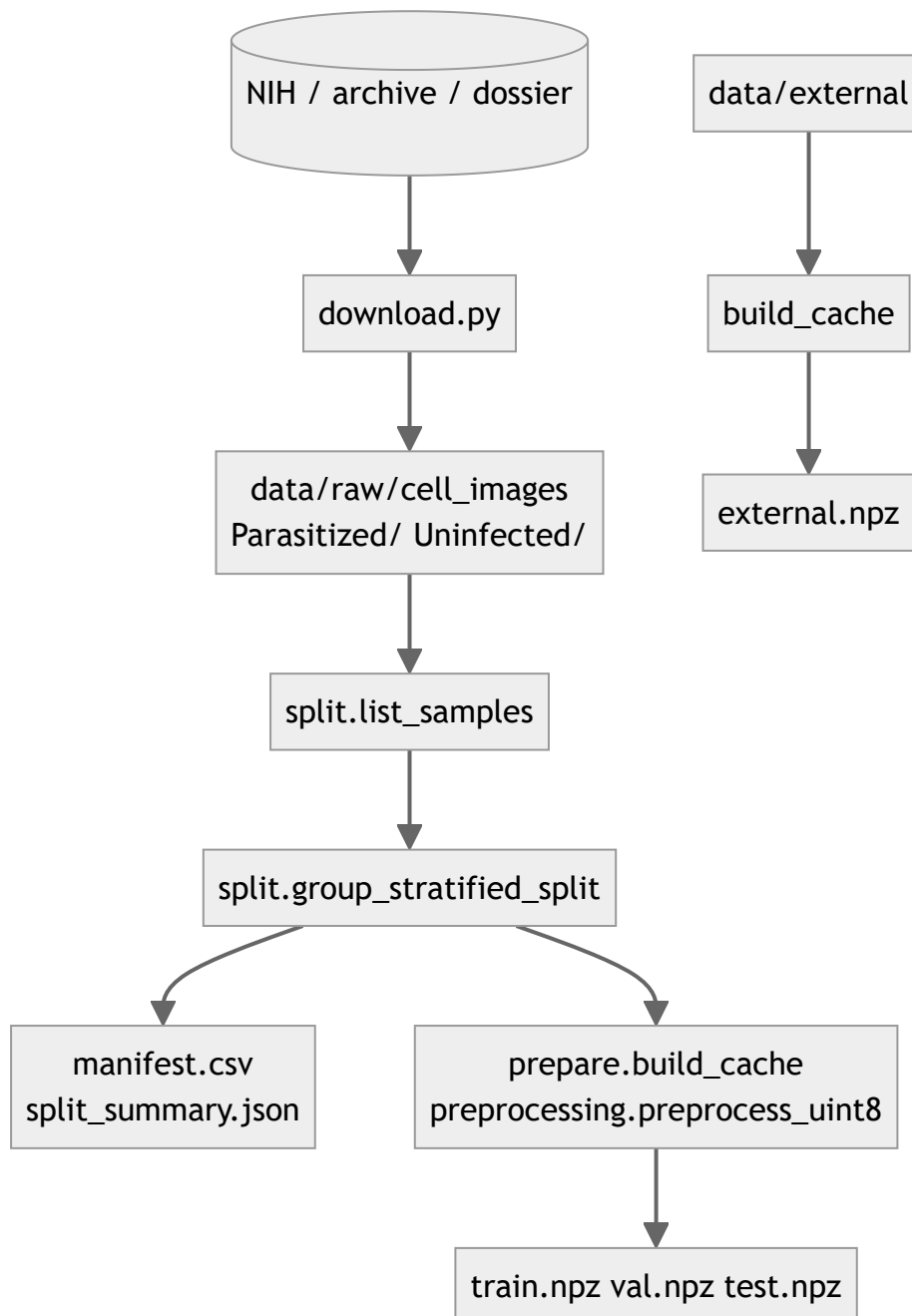
Classe Settings

Settings est une dataclass **gelée** (frozen=True). Les tests la construisent directement (Settings(model_path="unused.onnx", serve_web_app=False, rate_limit="1000/minute")) au lieu de passer par l'environnement. La propriété max_upload_bytes convertit max_upload_mb en octets.

Pipeline de données (src/data/)

Le pipeline transforme le dataset NIH brut en jeux `train` / `val` / `test` (+ `external`) prétraités et mis en cache. Il garantit deux propriétés :

- **Aucune fuite de patient** entre les splits (A4).
- **Prétraitement identique** entre l'entraînement et l'API (A2).



1. Dataset source

- Dataset **NIH Malaria Cell Images** : environ 27 558 images PNG de cellules segmentées, équilibrées entre `Parasitized` et `Uninfected`, issues d'environ 200 patients.
- Les noms de fichiers encodent le patient, par exemple `C100P61ThinF_IMG_20150918_144104_cell_162.png` → patient `C100P61ThinF`.
- Le fond des images NIH est **noir**. Le prétraitement en tient compte (composition des images transparentes sur fond noir).

Les fichiers `patientid_cellmapping_*.csv` à la racine ne sont pas utilisés par le code : l'identifiant patient est déduit du nom de fichier.

2. `download.py` — récupération (A1)

```
python -m src.data.download # téléchargement NIH
python -m src.data.download --source "Malaria Cell Images Dataset.zip"
python -m src.data.download --source "Malaria Cell Images Dataset/"
python -m src.data.download --dest autre/dossier
```

Fonction	Rôle
<code>find_class_root(base)</code>	Parcourt <code>base</code> et ses sous-dossiers, du moins profond au plus profond , et renvoie le premier qui contient à la fois <code>Parasitized/</code> et <code>Uninfected/</code> . Ignore ainsi le doublon imbriqué des archives Kaggle (<code>cell_images/cell_images</code>). <code>FileNotFoundError</code> sinon
<code>copy_dataset(src_root, dest)</code>	Copie les <code>.png</code> de chaque classe vers <code>dest/<classe>/</code> . N'écrase pas un fichier existant (relance idempotente). Renvoie le nombre d'images par classe
<code>download(url, target)</code>	Téléchargement HTTP en flux (<code>urllib</code>)
<code>main()</code>	Orchestration : téléchargement éventuel → extraction ZIP dans un dossier temporaire → copie normalisée

Les fichiers temporaires (archive téléchargée, extraction) sont supprimés à la fin (`tempfile.TemporaryDirectory`). Seuls les `.png` sont copiés (les `Thumbs.db` et autres sont ignorés).

3. `split.py` — split stratifié par patient (A4)

Pourquoi par patient

Un patient fournit de nombreuses cellules très semblables. Un split au niveau image placerait le même patient dans le train et le test. Le modèle « reconnaîtrait » alors le patient, et les métriques seraient surestimées. Le split répartit donc des **patients entiers**.

Sample

Dataclass gelée : `path` (str), `label` (int selon `class_map`), `patient_id` (str), `split` ("", `train`, `val`, `test` ou `external`).

Fonctions

Fonction	Rôle
<code>patient_id_from_filename(name)</code>	Préfixe avant <code>_IMG</code> ; à défaut, tout ce qui précède le dernier <code>_</code> (utile pour les lots du jeu externe <code>lab<lot>_<n>.png</code>)
<code>list_samples(root, class_map)</code>	Liste triée des <code>*.png</code> de <code>root/<classe>/</code> pour chaque classe de <code>class_map</code>
<code>group_stratified_split(samples, fractions, seed)</code>	Affectation des patients aux splits (voir algorithme)
<code>write_manifest(samples, path, root)</code>	CSV <code>path,label,patient_id,split</code> , chemins relatifs à <code>root</code> en notation POSIX
<code>read_manifest(path, root)</code>	Lecture inverse ; les chemins relatifs sont résolus depuis <code>root</code>
<code>split_summary(samples)</code>	Par split : nombre d'images, de patients, d'images par label

Algorithme de `group_stratified_split`

1. Vérifie que les fractions somment à 1 (tolérance `1e-6`), sinon `ValueError`.
2. Compte les images de chaque patient par classe : vecteur `counts[p] = [n_label0, n_label1]`.
3. Mélange les patients avec `numpy.random.default_rng(seed)`, puis les **trie par nombre d'images décroissant** (tri stable : à taille égale, l'ordre aléatoire est conservé). Placer les gros patients en premier améliore l'équilibre de l'algorithme glouton.
4. Calcule la cible `target[split, classe] = fraction(split) × total(classe)`.
5. Pour chaque patient, dans cet ordre : - déficit relatif $(\text{target} - \text{current}) / \max(\text{target}, 1)$ par split et par classe ; - score d'un split = $\sum_{\text{classes}} \text{déficit} \times \text{counts}[p]$; - le patient va dans le split au score maximal ; `current` est mis à jour.
6. Renvoie de nouveaux `Sample` avec le champ `split` renseigné.

Propriétés garanties (et testées dans `tests/unit/test_split.py`) :

- un patient n'apparaît que dans un seul split ;
- résultat **déterministe** pour une graine donnée, différent pour une autre graine ;
- proportions et équilibre des classes proches des cibles.

4. preprocessing.py — prétraitement partagé (A2)

Module **sans TensorFlow**, importé par l'API et par le pipeline. Toute modification change le comportement du modèle déployé : elle est couverte par des tests de valeurs de référence (`test_preprocessing_reference_values_are_stable`).

Contrat d'entrée du modèle

Propriété	Valeur
Espace couleur	RGB (transparence composée sur fond noir)
Taille	<code>image_size × image_size</code> (<code>128 × 128</code>), redimensionnement bilinéaire sans conserver le ratio
Type	<code>float32</code>
Plage	<code>[0, 1]</code> (division par 255)

Propriété	Valeur
Forme du batch	(N, 128, 128, 3) (NHWC)

La mise à l'échelle `[-1, 1]` propre à MobileNetV2 est une **couche du modèle** (`to_mobilenet_range`). Elle est donc embarquée dans l'ONNX : l'API n'a pas à la connaître.

API du module

Élément	Rôle
<code>ALLOWED_FORMATS</code>	<code>{"PNG", "JPEG"}</code>
<code>InvalidImageError(ValueError)</code>	Image illisible, corrompue ou de format refusé. Convertie en HTTP 400 par l'API
<code>QualityReport(ok, issues)</code>	Résultat du contrôle qualité
<code>decode_image(data, allowed_formats)</code>	Octets → image PIL RGB. Détecte le format réel (en-tête du fichier, pas l'extension), appelle <code>verify()</code> pour détecter les fichiers tronqués, puis recharge l'image (PIL interdit de réutiliser une image après <code>verify()</code>)
<code>load_image_file(path)</code>	Lecture disque + <code>decode_image</code>
<code>to_rgb(img)</code>	Images RGBA, LA ou P avec transparence : composition alpha sur fond noir opaque. Autres modes (L, CMYK...): <code>convert("RGB")</code>
<code>preprocess(img, size)</code>	Image → <code>float32 (size, size, 3)</code> dans <code>[0, 1]</code> . Utilisé par l'API
<code>preprocess_uint8(img, size)</code>	Même redimensionnement, stocké en <code>uint8</code> . Utilisé pour le cache (4× moins de mémoire)
<code>uint8_to_model_input(arr)</code>	<code>uint8</code> → <code>float32 / 255</code> . Garantit <code>uint8_to_model_input(preprocess_uint8(x)) == preprocess(x)</code>
<code>check_quality(img, min_side, min_std)</code>	IQA minimal : plus petit côté < <code>min_side</code> → « Image trop petite » ; écart-type des niveaux de gris < <code>min_std</code> → « Image quasi uniforme ». N'empêche pas la prédiction : les messages sont renvoyés dans <code>quality_warnings</code>

Point important : `train.py` et `OnnxPredictor.predict_uint8` refont la division par 255 sur le cache `uint8`. Le test `test_training_cache_matches_inference_preprocessing` vérifie que ce chemin donne exactement le même tenseur que `preprocess` dans l'API.

5. `prepare.py` — construction du cache

```
python -m src.data.prepare [--raw-dir data/raw/cell_images] [--out-dir data/processed]
```

Étapes de `main()` :

1. `list_samples` sur `raw_dir`. Sans image : arrêt avec un message qui renvoie vers `download`.
2. `group_stratified_split` avec `data.split` et `seed`.
3. Écriture de `manifest.csv` (traçabilité image → patient → split).
4. Pour chaque split, `build_cache` : décodage + `preprocess_uint8`, puis `np.savez(x, y)`. - `x` : `uint8` (N, 128, 128, 3) ; `y` : `int64` (N,) selon `class_map`. - Une image illisible est **ignorée** et comptée (`skipped`), avec un avertissement.

5. Jeu externe : si `data/external/Parasitized/` et `data/external/Uninfected/` existent, construction de `external.npz` . Avertissement si moins de 200 images (exigence A6), ou si le jeu est absent.
6. Écriture de `split_summary.json` .

Fichiers produits (`data/processed/` , hors Git)

Fichier	Contenu
<code>manifest.csv</code>	<code>path, label, patient_id, split</code> pour chaque image
<code>train.npz</code> , <code>val.npz</code> , <code>test.npz</code>	Clés <code>x</code> (uint8) et <code>y</code> (int64)
<code>external.npz</code>	Jeu externe, si présent
<code>split_summary.json</code>	Par split : <code>images</code> , <code>patients</code> , <code>label_0</code> , <code>label_1</code> , <code>skipped</code> (+ <code>external</code>)

Mémoire : le cache complet tient en environ 1,4 Go ($27\,558 \times 128 \times 128 \times 3$ octets). Il est chargé entièrement en RAM par `train.py` .

Modèle (`src/models/`)

Ce document couvre le cycle de vie du modèle : architecture, entraînement, export ONNX, calibration du seuil, évaluation, puis inférence et explicabilité.

Module	Dépend de TensorFlow	Utilisé par l'API
<code>model.py</code>	oui	non
<code>train.py</code>	oui	non
<code>export.py</code>	oui	non
<code>calibrate_threshold.py</code>	non (ONNX Runtime)	non
<code>evaluate.py</code>	non (ONNX Runtime)	non
<code>inference.py</code>	non	oui
<code>explain.py</code>	non	oui
<code>metrics.py</code>	non (numpy pur)	non
<code>artifacts.py</code>	non	oui

1. Artefacts versionnés — `artifacts.py` (B8)

`ModelPaths(version)` centralise tous les chemins d'une version. Le constructeur refuse une version qui n'est pas au format semver `X.Y.Z` (`ValueError`).

Propriété	Chemin	Versionné dans Git
<code>keras</code>	<code>models/keras/model_v<v>.keras</code>	non
<code>onnx</code>	<code>models/model_v<v>.onnx</code>	non
<code>metadata</code>	<code>models/model_v<v>.json</code> (fiche modèle)	non
<code>training_log</code>	<code>logs/training_v<v>.csv</code> (base des noms <code>_phase1</code> / <code>_phase2</code>)	non
<code>baseline</code>	<code>metrics_baseline/<v>.json</code>	oui

Fonctions utilitaires :

- `metadata_path_for(onnx_path)` : même nom que l'ONNX, extension `.json`. L'API l'utilise pour trouver la fiche d'un `MODEL_PATH` quelconque.
- `read_json(path)`, `write_json(path, data)` (UTF-8, indentation 2, crée le dossier parent).
- `update_metadata(path, **fields)` : fusionne des champs dans une fiche existante (utilisé par la calibration).

Fiche modèle (`model_v<v>.json`)

Créée par `export.py`, complétée par `calibrate_threshold.py` :

```
{
  "model_version": "1.0.0",
  "created_at": "2026-09-29T18:00:00+00:00",
  "class_map": {"0": "Uninfected", "1": "Parasitized"},
  "positive_class": "Parasitized",
  "sigmoid_output": "P(class_map[1]) = P(Parasitized)",
  "input": {"name": "image", "shape": [null, 128, 128, 3], "dtype": "float32",
    "preprocessing": "RGB, resize bilinéaire, /255 -> [0,1] (src/data/preprocessing.py)"},
  "outputs": ["probability", "cam"],
  "size_mb": 9.1,
  "quantized_int8": false,
  "sha256": "...",
  "decision_threshold": 0.3127,
  "calibration": {
    "dataset": "validation",
    "calibrated_at": "...",
    "rule": "précision maximale sous contrainte rappel >= cible",
    "target_recall": 0.98,
    "val_recall": 0.9803, "val_precision": 0.95, "val_specificity": 0.94, "val_auc": 0.99,
    "val_size": 4100,
    "pr_curve": [{"threshold": 0.1, "recall": 0.99, "precision": 0.9, "specificity": 0.88}, "..."]
  }
}
```

Les valeurs numériques ci-dessus sont des exemples. `decision_threshold` vaut `null` tant que la calibration n'a pas été faite ; l'API refuse alors de se déclarer prête.

2. Architecture — `model.py` (3.2)

```
image (N,128,128,3) float32 [0,1]
→ Rescaling(x2, -1)          "to_mobilenet_range" → [-1, 1]
→ MobileNetV2 (ImageNet, sans tête) dernière couche "out_relu" (N,4,4,1280)
→ GlobalAveragePooling2D    "gap" (N,1280)
→ Dense(128, ReLU)          "head_dense"
→ Dropout(0.3)              "head_dropout"
→ Dense(1, sigmoïde)         "probability" (N,1) = P(Parasitized)
```

Élément	Rôle
<code>build_model(cfg, weights="imagenet")</code>	Construit le modèle <code>palunet</code> . Le backbone est créé avec <code>input_tensor=x</code> : ses couches sont « à plat » dans le modèle final, ce qui rend <code>out_relu</code> accessible pour Grad-CAM. Backbone entièrement gelé à la sortie
<code>NON_BACKBONE</code>	Noms des couches qui ne font pas partie du backbone (entrée, rescaling, tête)
<code>backbone_layers(model)</code>	Couches du modèle hors <code>NON_BACKBONE</code>
<code>set_backbone_trainable(model, trainable_last)</code>	Gèle tout le backbone sauf ses <code>trainable_last</code> dernières couches. Les <code>BatchNormalization</code> restent toujours gelées : avec de petits batchs, les dégeler détruirait les statistiques ImageNet
<code>build_inference_model(model)</code>	Modèle d'export à deux sorties : <code>probability</code> (N,1) et <code>cam</code> (N,h,w)

Grad-CAM analytique, sans gradient à l'inférence

Grad-CAM pondère chaque canal k de la dernière carte de features A par $\alpha_k = \text{moyenne}_{\{i,j\}} \partial z / \partial A[i,j,k]$, où z est le logit de la classe 1.

La tête est $z = w_2 \cdot \text{ReLU}(w_1^T \cdot \text{GAP}(A) + b_1) + b_2$. Le gradient est donc :

$$\partial z / \partial A[i,j,k] = \sum_u w_2[u] \cdot 1[h_u > 0] \cdot w_1[k,u] / (H \cdot W)$$

Il ne dépend pas de (i, j) : c'est directement α_k . `cam_fn` calcule cette expression avec de simples produits matriciels, puis `cam = \sum_k \alpha_k \cdot A[:, :, :, k]`. Le graphe ONNX produit donc la carte Grad-CAM **brute** (avant ReLU, orientée vers la classe 1) sans calcul de gradient, et ONNX Runtime suffit à l'inférence.

La carte finale est `ReLU(cam)` si la classe prédite est 1, `ReLU(-cam)` si c'est la classe 0 (voir `explain.class_activation`). À 128 px d'entrée, la résolution de la carte est 4×4 , ensuite agrandie en 128×128 .

Si la tête change (couche supplémentaire, autre activation), la formule n'est plus valable et `build_inference_model` doit être réécrit.

3. Entraînement — `train.py` (B1–B4)

```
python -m src.models.train # complet
python -m src.models.train --limit 2000 --epochs1 1 --epochs2 1 # smoke test
python -m src.models.train --deterministic # opérations déterministes (plus lent)
```

Option	Défaut	Rôle
<code>--version</code>	<code>model.version</code>	Version des artefacts produits
<code>--epochs1</code> , <code>--epochs2</code>	15, 10	Époques max par phase
<code>--batch-size</code>	64	Taille de batch
<code>--limit</code>	aucun	Sous-échantillon du train ; la validation est réduite à <code>max(limit // 4, 64)</code>
<code>--deterministic</code>	non	<code>tf.config.experimental.enable_op_determinism()</code>

Déroulement

- `keras.utils.set_random_seed(seed)`.
- Chargement de `train.npz` et `val.npz`. **Le test n'est jamais chargé** (A4).
- `make_dataset` : `tf.data` avec mélange (train uniquement), batch, conversion `/255` (identique à `uint8_to_model_input`), augmentation (train uniquement), `prefetch`.
- Phase 1** — extraction de features : backbone gelé, Adam lr $1e-4$.
- Phase 2** — fine-tuning : dégel des 30 dernières couches du backbone (hors BatchNorm), recompilation, Adam lr $1e-5$.

- Chaque phase : `EarlyStopping(val_loss, patience=4, restore_best_weights=True)` et `CSVLogger` vers `logs/training_v<v>_phase{1,2}.csv`.
- Sauvegarde `models/keras/model_v<v>.keras`.
- Métriques de validation **au seuil 0,5, à titre indicatif** (le seuil réel est calibré plus tard) et ajout d'une ligne dans `logs/experiments.csv` (E8).

Augmentation (`augmentation(cfg)`) : `RandomFlip("horizontal")` (si activé), `RandomRotation` ($\pm 20^\circ$), `RandomTranslation` ($\pm 10\%$), `RandomZoom` ($\pm 15\%$). Remplissage constant (noir), cohérent avec le fond des images NIH.

Métriques Keras suivies : `accuracy`, `auc`, `recall` et `precision` de la classe d'indice 1.

Colonnes de `logs/experiments.csv` : `timestamp`, `model_version`, `train_size`, `val_size`, `epochs1`, `epochs2`, `batch_size`, `lr1`, `lr2`, `val_accuracy@0.5`, `val_recall@0.5`, `val_auc`, `duration_s`.

4. Export ONNX — `export.py` (B5)

```
python -m src.models.export [--version 1.0.0] [--force-quantize]
```

- Charge le `.keras` et construit le modèle d'inférence (`probability` + `cam`).
- Conversion `tf2onnx` (opset 13), entrée nommée `image`, forme (`None`, 128, 128, 3).
- Vérification d'équivalence** Keras / ONNX Runtime sur 64 images de validation : écart absolu max de probabilité $\leq 1e-4$, sinon arrêt.
- Si l'ONNX dépasse `export.max_size_mb` (20 Mo) ou si `--force-quantize` : la version fp32 est renommée `model_v<v>_fp32.onnx`, puis une quantification dynamique int8 (`QUInt8`) produit le nouvel `model_v<v>.onnx`. Nouvelle vérification avec une tolérance de 0,05.
- Écriture de la fiche modèle (`decision_threshold: null`, sha256 de l'artefact final).

Après l'export, il **faut** lancer la calibration : sans elle, l'API reste non prête.

5. Moteur d'inférence — `inference.py`

Partagé par l'API, la calibration et l'évaluation. Ainsi, le seuil est calibré et les métriques sont mesurées sur l'artefact **réellement déployé** (le `.onnx`), pas sur le modèle Keras.

Élément	Rôle
<code>RawOutput(sigmoid, cam)</code>	<code>sigmoid</code> : (N,) = P(classe 1). <code>cam</code> : (N,h,w) brut ou <code>None</code>
<code>Predictor</code> (Protocol)	Interface : attribut <code>has_cam</code> , méthode <code>run(batch, with_cam)</code>
<code>OnnxPredictor(model_path, intra_op_threads, session)</code>	Session ONNX Runtime sur <code>CPUExecutionProvider</code> . Le paramètre <code>session</code> permet d'injecter une fausse session dans les tests
<code>OnnxPredictor._pick(outputs, name, rank)</code>	Trouve une sortie par nom exact, par préfixe <code>name</code> : (noms générés par <code>tf2onnx</code>), ou à défaut par rang (2 pour <code>probability</code> , 3 pour <code>cam</code>). Sans sortie <code>probability</code> : <code>ValueError</code>
<code>OnnxPredictor.run(batch, with_cam)</code>	Ne demande la sortie <code>cam</code> que si elle est voulue et disponible (gain de temps)
<code>OnnxPredictor.predict_uint8(x, batch_size=256)</code>	Inférence par lots sur un cache uint8 (évaluation hors ligne)
<code>MockPredictor</code>	Prédicteur factice déterministe (voir ci-dessous)

Élément	Rôle
<code>Decision</code>	<code>prediction</code> , <code>confidence</code> , <code>probability_parasitized</code> , <code>predicted_index</code>
<code>decide(sigmoid, threshold, class_map, positive_class)</code>	Règle de décision

Règle de décision

```
p_pos = positive_probability(sigmoid)          # probabilité de Parasitized
si p_pos >= seuil : prediction = Parasitized, confidence = p_pos
sinon              : prediction = Uninfected, confidence = 1 - p_pos
probability_parasitized = p_pos (toujours)
```

`confidence` est la probabilité de la classe **prédite**, pas celle d'une classe fixe. Avec un seuil bas (ex. 0,3), une cellule à `p_pos = 0,35` est déclarée parasitée avec une confiance de 0,35 : c'est voulu, le rappel est prioritaire.

MockPredictor

Heuristique **sans valeur médicale**, pour développer le front et le back contre le contrat figé :

- pixels de cellule : niveau de gris moyen > 0,05 (le fond noir est ignoré) ;
- pixels « violets » : bleu > rouge à l'intérieur de la cellule ;
- `sigmoid = 1 / (1 + exp(-200 * (fraction_violette - 0,02)))` ;
- `cam` : fraction violette moyenne sur une grille 4 × 4, moins 0,01. Suppose une image de côté multiple de 4 (128 px).

6. Explicabilité — `explain.py` (C9 / D6)

Post-traitement numpy de la sortie `cam`.

Fonction	Rôle
<code>class_activation(cam_raw, predicted_index)</code>	Garde la partie qui soutient la classe prédite (<code>cam</code> si classe 1, <code>-cam</code> si classe 0), applique ReLU, normalise par le maximum dans <code>[0, 1]</code>
<code>render_heatmap(activation, size, max_alpha=0.6)</code>	Agrandissement bilinéaire en <code>size × size</code> , palette type « jet » (bleu → cyan → jaune → rouge), canal alpha proportionnel à l'activation (max 60 %)
<code>heatmap_base64(cam_raw, predicted_index, size)</code>	PNG RGBA encodé en base64 : champ <code>heatmap_base64</code> de l'API, à superposer à l'image

7. Métriques — `metrics.py`

numpy pur, utilisable partout. Convention : `y_true` vrai pour la classe positive, `scores` = probabilité de la classe positive, décision positive si `score >= seuil`.

Fonction	Rôle
<code>roc_auc(y_true, scores)</code>	AUC par la statistique de Mann-Whitney, rangs moyens pour les ex-aequo. <code>nan</code> si une seule classe est présente
<code>confusion(y_true, y_pred)</code>	{tp, fn, fp, tn}

Fonction	Rôle
<code>classification_metrics(y_true, scores, threshold)</code>	<code>n</code> , <code>threshold</code> , <code>accuracy</code> , <code>recall</code> , <code>precision</code> , <code>specificity</code> , <code>f1</code> , <code>auc</code> , <code>confusion_matrix</code> . Divisions par zéro → 0
<code>choose_threshold(y_true, scores, target_recall)</code>	Calibration (B6), voir ci-dessous
<code>check_against_baseline(metrics, baseline, tolerance)</code>	Liste des régressions sur <code>accuracy</code> , <code>recall</code> , <code>auc</code> (métrique < baseline – tolérance)
<code>check_acceptance(metrics, acceptance)</code>	Liste des critères absolus non atteints (<code>accuracy</code> ≥ 0,95, <code>recall</code> ≥ 0,96, <code>auc</code> ≥ 0,97)

Algorithme de `choose_threshold`

1. Trie les scores par ordre décroissant et cumule TP et FP.
2. Les seuils candidats sont les **valeurs distinctes** des scores (on garde la dernière position de chaque valeur, pour traiter correctement les ex-aequo).
3. Pour chaque candidat : $\text{rappel} = \text{TP} / P$, $\text{précision} = \text{TP} / (\text{TP} + \text{FP})$.
4. Parmi les candidats où $\text{rappel} \geq \text{cible}$, retient la **précision maximale** ; à précision égale, le **seuil le plus élevé**.
5. `ValueError` s'il n'y a aucun positif. La cible est toujours atteignable en théorie (le plus petit score donne un rappel de 1).

8. Calibration — `calibrate_threshold.py` (B6)

```
python -m src.models.calibrate_threshold [--version 1.0.0] [--target-recall 0.98]
```

1. Charge `val.npz` (**jamais le test**) et score avec l'`OnnxPredictor` de l'artefact exporté.
2. `choose_threshold` avec la cible de rappel.
3. Arrondit le seuil **vers le bas** à 4 décimales : un seuil plus bas ne peut que conserver ou augmenter le rappel.
4. Écrit `decision_threshold` et un bloc `calibration` (règle, cible, rappel / précision / spécificité / AUC de validation, taille, points de la courbe précision-rappel aux seuils 0,1 ... 0,9) dans la fiche modèle.

La variable `DECISION_THRESHOLD` de l'API peut surcharger ce seuil en exploitation.

9. Évaluation — `evaluate.py` (B7 / E4 / E5)

```
python -m src.models.evaluate # test (+ externe si présent)
python -m src.models.evaluate --write-baseline # fige metrics_baseline/<v>.json
python -m src.models.evaluate --check-baseline # non-régression
python -m src.models.evaluate --images-dir audit/2026-10 # audit de dérive mensuel
```

Fonction	Rôle
<code>score_arrays(predictor, x_uint8, y, cfg)</code>	→ (<code>y_pos</code> , <code>p_pos</code>) orientés vers la classe positive
<code>load_npz(name, cfg)</code>	(<code>x</code> , <code>y</code>) d'un cache, ou <code>None</code> s'il n'existe pas
<code>load_folder(folder, cfg)</code>	Charge et prétraite un dossier <code>Parasitized/</code> + <code>Uninfected/</code> (images illisibles ignorées)

Comportement :

- Arrêt si la fiche modèle n'a pas de seuil calibré.
- Évalue `test` (obligatoire) et `external` (si présent) avec le **seuil calibré**, affiche le JSON des métriques, avertit pour chaque critère d'acceptation non atteint.
- `--write-baseline` : écrit `metrics_baseline/<v>.json` (version, date, seuil, tolérance, métriques test arrondies, taille du test, métriques externes, `acceptance_passed`).
- `--check-baseline` : compare le test à la baseline.
- `--images-dir` : évalue un dossier étiqueté et compare le rappel à `drift_alert_recall`.

Code de sortie	Signification
0	OK
1	Régression par rapport à la baseline (<code>--check-baseline</code>)
2	Alerte de dérive : rappel < 0,94 (<code>--images-dir</code>)

Ces codes permettent d'intégrer l'évaluation à un pipeline CI ou à une tâche planifiée.

API d'inférence (`src/api/`)

API FastAPI servie par Uvicorn. Elle charge un modèle ONNX versionné, valide les images reçues, applique le seuil calibré et renvoie une prédiction, avec en option une carte Grad-CAM. Elle ne dépend pas de TensorFlow.

```
uvicorn src.api.main:app --port 8000 # modèle réel
MOCK_MODEL=true uvicorn src.api.main:app --port 8000 # sans modèle (développement)
```

- Interface de démonstration : `http://localhost:8000/`
- OpenAPI / Swagger : `http://localhost:8000/docs` (ReDoc : `/redoc`)

1. Organisation

Fichier	Responsabilité
<code>main.py</code>	Fabrique <code>create_app()</code> , cycle de vie, middleware d'observation, <code>RequestMetrics</code> , montage des routes et de <code>web_app/</code>
<code>settings.py</code>	<code>Settings</code> : configuration par variables d'environnement (voir configuration.md)
<code>dependencies.py</code>	Injection du service, authentification par clé API, clé de rate limiting, construction du <code>Limiter</code>
<code>service.py</code>	<code>ModelService</code> : chargement et contrôle du modèle, préparation des images, prédiction, journalisation
<code>schemas.py</code>	Modèles Pydantic du contrat <code>/v1</code> (figé)
<code>routes/health.py</code>	<code>/health</code> , <code>/health/live</code> , <code>/metrics</code>
<code>routes/predict.py</code>	<code>/v1/predict</code> , <code>/v1/predict/batch</code>

2. Cycle de vie de l'application — `main.py`

`create_app(settings=None, service=None)` :

1. Lit `Settings.from_env()` si aucun `settings` n'est fourni.
2. Configure les logs JSON (`configure_logging`).
3. Crée l'application avec le titre, la version d'API `1.0.0` et l'avertissement médical comme description OpenAPI.
4. Stocke dans `app.state` : `settings` , `service` (éventuellement injecté par les tests), `metrics` (`RequestMetrics`), `limiter` .
5. Enregistre le gestionnaire `RateLimitExceeded` (réponse 429 de slowapi).
6. Ajoute le middleware CORS si `CORS_ORIGINS` est défini (méthodes `GET` , `POST` ; en-têtes `X-API-Key` , `Content-Type`).
7. Ajoute le middleware `observe` (latence, codes HTTP, filet de sécurité 500).
8. Monte les routeurs `health` et `predict` , puis `web_app/` en fichiers statiques sur `/` (`html=True` : `/` sert `index.html`).

Au démarrage (`lifespan`), si aucun service n'a été injecté :

- `ModelService.from_settings(settings)` charge le modèle ;

- en cas de `ModelLoadError`, l'instance **reste vivante** (`/health/live` = 200) mais **non prête** (`/health` = 503 avec le message d'erreur, routes `/v1/*` = 503). L'orchestrateur ne lui envoie donc pas de trafic ;
- si `API_KEY` est vide, un avertissement rappelle que l'authentification est désactivée.

Le module expose `app = create_app()` pour Uvicorn. Les tests appellent `create_app()` avec des `Settings` et un service mock.

Middleware `observe`

- Mesure la durée de chaque requête.
- Capture toute exception non gérée : log `Erreur interne` avec la trace, réponse `500 {"detail": "Erreur interne du serveur"}` sans détail technique pour le client.
- Enregistre code HTTP et latence dans `RequestMetrics`, **uniquement pour les chemins** `/v1/*`.

`RequestMetrics`

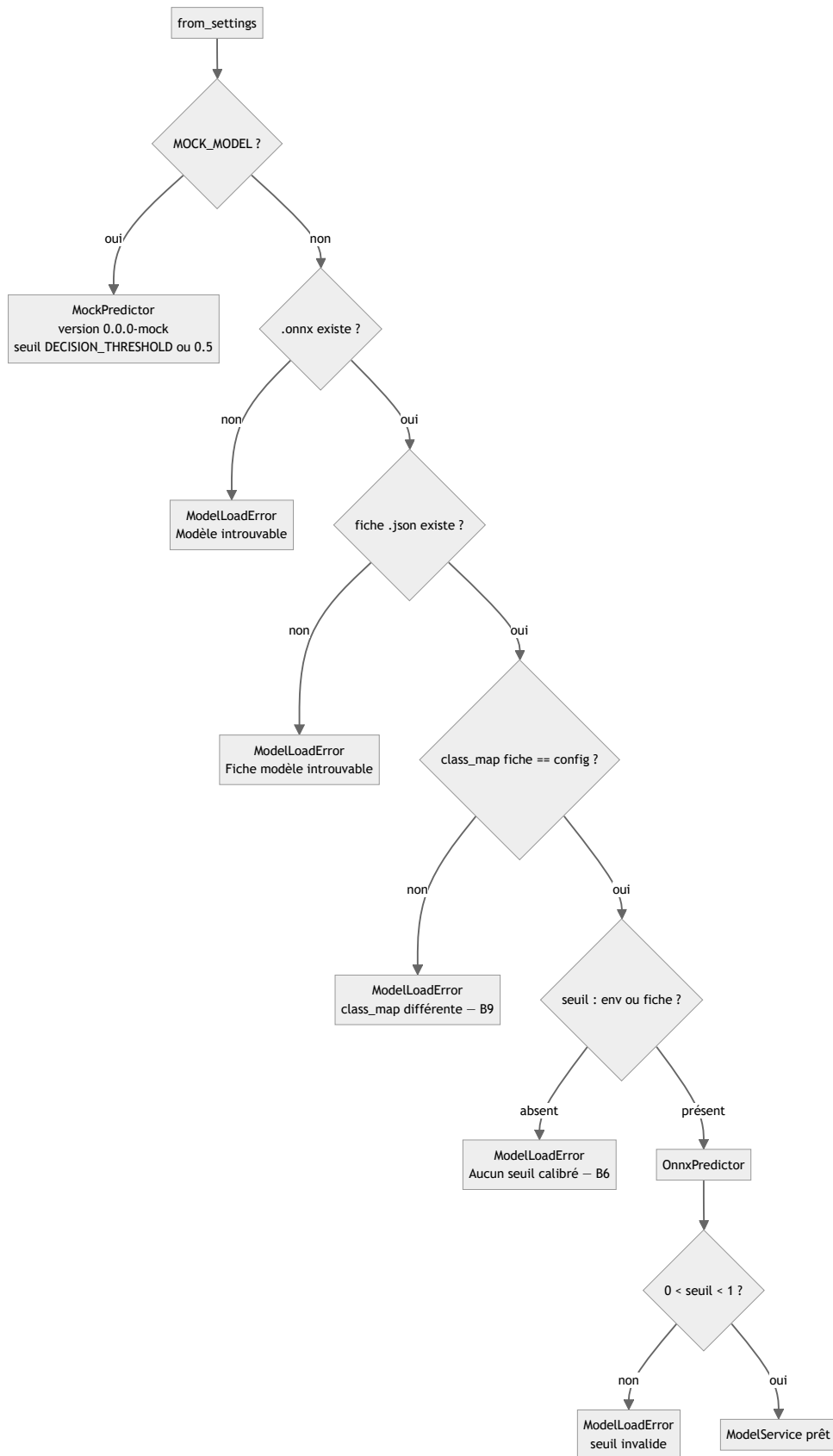
Compteurs en mémoire, propres à chaque instance, protégés par un verrou :

- `latencies` : les 1 000 dernières latences (fenêtre glissante) ;
- `status_counts` : nombre de réponses par code HTTP depuis le démarrage.

`snapshot()` renvoie `requests_total`, `error_rate` (part des codes ≥ 400), `status_codes` et les percentiles `p50`, `p95`, `p99` (méthode du rang le plus proche sur la fenêtre).

3. Service de prédiction — service.py

ModelService.from_settings(settings)



Paramètres communs lus dans `config.yaml` : `class_map`, `positive_class`, `image_size`, `min_side_px`, `min_std`.

Méthodes

Méthode	Rôle
<code>prepare(data: bytes) -> PreparedImage</code>	<code>decode_image</code> (format réel PNG/JPEG, détection de corruption), <code>check_quality</code> , <code>preprocess</code> . Lève <code>InvalidImageError</code>
<code>predict(images, include_heatmap=False, endpoint="predict") -> list[dict]</code>	Empile les images en un seul batch , une seule inférence ONNX, applique <code>decide()</code> à chaque image, génère la carte Grad-CAM si demandée et disponible, journalise chaque prédiction
<code>has_heatmap</code>	Vrai si le prédicteur expose une sortie <code>cam</code>
<code>health()</code>	Contenu de <code>/health</code> quand le service est prêt
<code>_log(result, endpoint)</code>	Journal de prédiction anonymisé (C5)

`processing_time_ms` est le temps d'**inférence** du batch (même valeur pour toutes les images d'un lot), pas la durée totale de la requête HTTP.

Journal de prédictions anonymisé (C5)

Chaque prédiction produit un enregistrement sur le logger `palunet.predictions` :

```
{"timestamp": "2026-10-05T10:12:03.120000+00:00", "level": "INFO",
  "logger": "palunet.predictions", "message": "prediction",
  "prediction_id": "3f1c...", "endpoint": "predict", "prediction": "Parasitized",
  "probability_parasitized": 0.9421, "decision_threshold": 0.3127,
  "model_version": "1.0.0", "processing_time_ms": 21, "quality_warnings": 0}
```

Ne sont **jamais** journalisés : le nom de fichier (les noms NIH contiennent l'identifiant patient), l'image, l'adresse IP, la clé API. Ce point est vérifié par `test_prediction_log_is_anonymised`.

4. Sécurité et quotas — `dependencies.py`

Élément	Comportement
<code>get_settings(request)</code>	Renvoie <code>app.state.settings</code>
<code>get_service(request)</code>	Renvoie le service, ou 503 Modèle non chargé
<code>verify_api_key</code>	Dépendance de toutes les routes <code>/v1/*</code> . Si <code>API_KEY</code> est vide : aucun contrôle. Sinon, compare l'en-tête <code>X-API-Key</code> à chaque clé autorisée en temps constant (<code>hmac.compare_digest</code>), sinon 401
<code>rate_limit_key(request)</code>	Clé du quota : <code>key:<X-API-Key></code> si l'en-tête est présent, sinon <code>ip:<adresse></code>
<code>build_limiter(settings)</code>	<code>slowapi.Limiter</code> avec <code>RATE_LIMIT_STORAGE</code>

Les routes de santé (`/health`, `/health/live`, `/metrics`) sont **publiques** : elles servent aux sondes de l'orchestrateur.

Ordre des contrôles sur `/v1/*` : authentification (dépendance FastAPI) → quota (décorateur `slowapi`) → service disponible → validation du fichier. Une clé invalide reçoit donc 401 sans consommer de quota. Chaque **requête** compte pour 1 dans le quota, y compris un lot de 32 images.

5. Référence des routes

POST /v1/predict

Requête `multipart/form-data` :

Champ	Type	Obligatoire	Description
<code>file</code>	fichier	oui	Image PNG ou JPEG d'une cellule unique, ≤ 5 Mo
<code>include_heatmap</code>	bool	non	Renvoyer la carte Grad-CAM. Accepté en champ de formulaire ou en paramètre de requête <code>?include_heatmap=true</code>

En-tête : `X-API-Key` si l'authentification est active.

```
curl -H "X-API-Key: $API_KEY" -F file=@cellule.png -F include_heatmap=true \
http://localhost:8000/v1/predict
```

Réponse 200 (`PredictionResponse`) :

Champ	Type	Description
<code>prediction</code>	str	<code>Parasitized</code> ou <code>Uninfected</code>
<code>confidence</code>	float [0,1]	Probabilité de la classe prédite
<code>probability_parasitized</code>	float [0,1]	Probabilité d'infection, quelle que soit la prédiction
<code>model_version</code>	str	Version du modèle servi
<code>processing_time_ms</code>	int	Durée de l'inférence
<code>heatmap_base64</code>	str null	PNG RGBA 128 × 128 en base64 (si demandé et disponible)
<code>quality_warnings</code>	list[str]	Avertissements IQA ; la prédiction est quand même fournie

```
{
  "prediction": "Parasitized",
  "confidence": 0.9421,
  "probability_parasitized": 0.9421,
  "model_version": "1.0.0",
  "processing_time_ms": 21,
  "heatmap_base64": null,
  "quality_warnings": []
}
```

POST /v1/predict/batch

Champ	Type	Description
<code>files</code>	fichiers (répété)	1 à 32 images PNG/JPEG, ≤ 5 Mo chacune

- Plus de `MAX_BATCH_SIZE` fichiers : **400** pour toute la requête.
- Un fichier invalide n'invalide **pas** le lot : il apparaît dans `errors` avec son code (400 ou 413) ; les autres sont prédits en une seule inférence.
- Pas de carte Grad-CAM en mode lot.
- Ici le nom de fichier est renvoyé au client (pour faire le lien), mais il n'est pas journalisé.

Réponse 200 (BatchResponse) :

```
{
  "results": [
    {
      "filename": "a.png",
      "prediction": "Parasitized",
      "confidence": 0.97,
      "probability_parasitized": 0.97,
      "model_version": "1.0.0",
      "processing_time_ms": 35,
      "heatmap_base64": null,
      "quality_warnings": []
    }
  ],
  "errors": [
    {
      "filename": "broken.png",
      "status_code": 400,
      "detail": "Fichier image illisible ou corrompu"
    }
  ],
  "model_version": "1.0.0",
  "processing_time_ms": 41
}
```

Le `processing_time_ms` global inclut la lecture et la validation des fichiers.

GET /health (readiness)

- 200: {"status": "ok", "model_version", "decision_threshold", "class_map", "heatmap_available"}
- 503: {"status": "unavailable", "detail": "<raison du non-chargement>"}

GET /health/live (liveness)

Toujours 200 {"status": "alive"} tant que le processus répond.

GET /metrics

```
{
  "requests_total": 120,
  "error_rate": 0.0083,
  "status_codes": {"200": 119, "400": 1},
  "latency_ms": {"p50": 18.2, "p95": 41.0, "p99": 77.5, "window": 120},
  "model_version": "1.0.0",
  "decision_threshold": 0.3127
}
```

Métriques **par instance**, remises à zéro au redémarrage. Format JSON, pas Prometheus.

Codes d'erreur

Code	Cause	Corps
400	Format autre que PNG/JPEG, fichier corrompu ou tronqué, fichier vide, lot trop grand	{"detail": "..."}
401	X-API-Key absent ou invalide	{"detail": "..."}
413	Fichier > MAX_UPLOAD_MB	{"detail": "Fichier trop volumineux (max 5 Mo)"}
422	Champ file / files manquant (validation FastAPI)	{"detail": [...]}
429	Quota dépassé	{"error": "Rate limit exceeded: ..."} (format slowapi)
503	Modèle non chargé	{"detail": "Modèle non chargé"}

Code	Cause	Corps
500	Erreur interne	<code>{"detail": "Erreur interne du serveur"}</code>

La taille est contrôlée en lisant au plus `max_bytes + 1` octets : un fichier trop gros n'est jamais chargé entièrement en mémoire.

6. Contrat et versionnement

Le contrat `/v1` défini dans `schemas.py` est **figé**. Ajouter un champ optionnel reste compatible. Renommer, supprimer ou changer le sens d'un champ impose un nouveau préfixe `/v2`. La version d'API (`API_VERSION = "1.0.0"`) est distincte de la version du modèle (`model_version`).

7. Concurrence et performance

- Les routes sont des fonctions **synchrones** (`def`) : FastAPI les exécute dans son pool de threads, et l'inférence ONNX ne bloque pas la boucle d'événements.
- Une session ONNX Runtime unique est partagée entre les threads (son `run` est thread-safe).
- `INTRA_OP_THREADS` limite le parallélisme interne d'ONNX Runtime : l'aligner sur les vCPU évite la sur-souscription (Dockerfile : 2).
- Docker lance **un seul worker** Uvicorn. Pour monter en charge, multiplier les conteneurs et partager le quota via `RATE_LIMIT_STORAGE=redis://...`.
- SLO : P95 de `/v1/predict` < 500 ms sur 2 vCPU / 2 Go (vérifié par le test de charge Locust).

Interface web de démonstration (`web_app/`)

Page statique sans framework ni étape de build, servie par FastAPI sur `/` (même origine que l'API, donc pas de CORS nécessaire). Désactivable avec `SERVE_WEB_APP=false`.

Fichier	Rôle
<code>index.html</code>	Structure : bandeau d'avertissement, zone de dépôt, options, résultat, erreurs
<code>app.js</code>	Logique : santé de l'API, sélection et contrôle du fichier, appel <code>/v1/predict</code> , rendu
<code>styles.css</code>	Mise en forme (carte, barre de confiance, superposition Grad-CAM)

1. Parcours utilisateur

- Au chargement, `checkHealth()` appelle `GET /health` et affiche : - « API opérationnelle — modèle vX, seuil de décision Y » (statut `ok`) ; - ou « API indisponible : » (statut `ko`). Si `heatmap_available` est faux, la case Grad-CAM est désactivée.
- L'utilisateur dépose une image (glisser-déposer) ou la choisit.
- `selectFile()` contrôle côté client le type MIME (`image/png`, `image/jpeg`) et la taille (≤ 5 Mo). Le serveur refait ces contrôles : ceux du client servent seulement à répondre plus vite.
- Clic sur « Analyser » : `submit()` envoie un `FormData` (`file`, `include_heatmap`) avec l'en-tête `X-API-Key` s'il est renseigné. Le bouton est désactivé pendant l'analyse.
- `render()` affiche le résultat.

2. Affichage du résultat

Élément	Source	Remarque
Classe prédite	<code>prediction</code>	Libellé français : « Parasitée » (rouge) / « Saine » (vert)
Libellé du score	<code>prediction</code>	« Confiance que la cellule est « parasitée » » : le libellé précise la classe concernée (exigence D2)
Barre et pourcentage	<code>confidence</code>	Format français (<code>94,2 %</code>)
Probabilité d'infection	<code>probability_parasitized</code>	Toujours affichée, quelle que soit la classe
Avertissements	<code>quality_warnings</code>	Préfixés « Qualité d'image : »
Carte Grad-CAM	<code>heatmap_base64</code>	Image PNG RGBA superposée à l'aperçu
Méta	<code>model_version</code> , <code>processing_time_ms</code>	

L'aperçu de l'image est créé localement (`URL.createObjectURL`) : l'image n'est pas renvoyée par le serveur.

3. Messages d'erreur (D3)

`describeError(status, detail)` traduit les codes HTTP en messages compréhensibles :

Code	Message
400	« Image refusée : »
401	« Accès refusé : clé API manquante ou invalide »
413	« Fichier trop volumineux : la taille maximale est de 5 Mo. »
429	« Trop de requêtes : merci de patienter une minute... »
503	« Le service d'analyse n'est pas prêt (modèle non chargé)... »
autre	« Erreur inattendue du serveur ()... »
réseau	« Impossible de joindre l'API. Vérifiez votre connexion. »

4. Clé API

Le champ « Clé API » (repliable) est mémorisé dans `localStorage` (`palunet_api_key`). Les accès à `localStorage` sont protégés par `try/catch` (navigation privée, stockage bloqué).

La clé est stockée en clair dans le navigateur. Acceptable pour une démonstration sur poste maîtrisé ; à revoir avant tout usage multi-utilisateur (authentification par session, par exemple).

5. Exigences d'interface couvertes

Exigence	Mise en œuvre
D5 — avertissement visible	Bandeau <code>.disclaimer</code> en haut de page
D2 — score non ambigu	Libellé dynamique du score selon la classe prédite
D3 — erreurs compréhensibles	<code>describeError()</code>
D6 — explicabilité	Case Grad-CAM, superposition de <code>heatmap_base64</code>
Confidentialité	Pied de page : aucune image conservée, journaux anonymisés

6. Modifier l'interface

- `API_BASE = ""` : l'API est sur la même origine. Pour servir l'interface ailleurs, renseigner l'URL de l'API ici **et** `CORS_ORIGINS` côté serveur.
- `MAX_BYTES` doit rester aligné sur `MAX_UPLOAD_MB`.
- Les libellés de classes sont dans `LABELS_FR`. Une nouvelle classe s'affiche sous son nom anglais tant qu'elle n'y est pas ajoutée.

Tests et qualité (tests/)

```
pytest --cov                # unitaires + intégration (+ non-régression si le modèle existe)
pytest tests/unit -q        # unitaires seuls
pytest -k batch             # filtrer par nom
```

Configuration dans `pyproject.toml` : `testpaths = ["tests"]`, `pythonpath = ["."]` (imports `src.*` sans installation), `-q`, avertissements de dépréciation masqués.

État au 5 octobre 2026 (Python 3.11.9, Windows) : **67 tests passés, 4 ignorés** (non-régression du modèle, faute de modèle entraîné), **couverture 94 %** (seuil exigé : 70 %).

1. Organisation

Dossier	Type	Dépendances	Exécution
tests/unit/	Unitaires	numpy, PIL	Toujours, en CI
tests/integration/	API de bout en bout via <code>TestClient</code> , avec <code>MockPredictor</code>	FastAPI	Toujours, en CI
tests/model/	Non-régression du vrai modèle ONNX	modèle + baseline + <code>test.npz</code>	Ignorés automatiquement si absents
tests/load/	Charge (Locust)	API démarrée	Manuel

Aucun test ne requiert TensorFlow : la CI installe seulement `requirements-dev.txt`.

2. Fixtures — tests/conftest.py

Élément	Rôle
<code>make_cell(infected, size=140, seed=0)</code>	Cellule synthétique : disque rose sur fond noir, bruit gaussien, et 3 taches violettes si infectée. Le <code>MockPredictor</code> la classe correctement
<code>to_bytes(img, fmt="PNG")</code>	Image PIL → octets
<code>cfg</code>	<code>load_config()</code>
<code>infected_png, healthy_png</code>	Octets PNG des deux cellules types
<code>mock_service(cfg, threshold=0.5)</code>	<code>ModelService</code> avec <code>MockPredictor</code> , version 1.0.0
<code>make_client(cfg, **overrides)</code>	<code>TestClient</code> sur <code>create_app(Settings(...), service=mock_service)</code> , sans interface web. Les <code>overrides</code> modifient <code>Settings</code> (clés, quota, taille max...)
<code>client</code>	Client standard avec un quota de 1000/minute

3. Couverture fonctionnelle

Unitaires

Fichier	Ce qui est vérifié
test_preprocessing.py	Forme, type et plage de <code>preprocess</code> ; égalité exacte entre le cache uint8 et le prétraitement de l'API ; valeurs de référence figées (détecte tout changement involontaire) ; PNG et JPEG acceptés ; autres formats, données corrompues et PNG tronqué refusés ; composition RGBA sur fond noir ; niveaux de gris → RGB ; IQA (petite image, image uniforme) ; le format réel prime sur l'extension
test_split.py	Extraction de l'identifiant patient ; aucun patient dans deux splits ; proportions et équilibre des classes ; reproductibilité selon la graine ; fractions dont la somme $\neq 1$ refusées ; aller-retour du manifeste
test_metrics.py	AUC parfaite / aléatoire / inversée ; AUC égale à la définition par paires ; AUC <code>nan</code> sur une seule classe ; matrice de confusion ; <code>choose_threshold</code> atteint la cible de rappel avec la meilleure précision ; contrôles baseline et acceptation
test_inference.py	<code>class_map</code> figée ; validation de <code>class_map</code> ; <code>positive_probability</code> avec encodage normal et inversé ; <code>decide</code> (positif, négatif, confiance de la classe prédite, seuil calibré et non 0,5) ; <code>OnnxPredictor</code> avec fausse session (avec / sans <code>cam</code> , sortie <code>probability</code> obligatoire) ; <code>MockPredictor</code> sépare les cellules synthétiques ; signe de la carte selon la classe ; rendu PNG de la heatmap

Intégration (`tests/integration/test_api.py`)

Thème	Tests
Santé	Contrat <code>/health</code> ; 503 et <code>/v1</code> en 503 si le modèle manque, liveness toujours 200
Prédiction	Contrat de réponse ; <code>confidence</code> relative à la classe prédite ; heatmap par formulaire et par paramètre de requête ; JPEG accepté ; avertissements qualité
Erreurs	Fichiers invalides (400) ; format non supporté (400) ; trop gros (413) ; fichier manquant (422)
Sécurité	Clé API exigée si configurée, plusieurs clés, <code>/health</code> public ; quota par clé (la 4 ^e requête à 3/minute → 429, une autre clé passe)
Lot	Échec partiel (résultats + erreurs) ; plus de 32 fichiers → 400
Observabilité	Journal anonymisé (ni nom de fichier ni IP) ; <code>/metrics</code> (total, taux d'erreur, p95, version)
Chargement	Lecture de la fiche et surcharges env ; refus si <code>class_map</code> inversée, seuil absent ou hors]0,1[; fiche manquante ; mode mock ; lecture des variables d'environnement

Non-régression du modèle (`tests/model/test_model_regression.py` , E1/E4)

Exécutés seulement si `models/model_v<v>.onnx`, sa fiche, `metrics_baseline/<v>.json` et `data/processed/test.npz` existent :

- pas de régression d' `accuracy` , `recall` ou `auc` au-delà de la tolérance (0,005) ;
- **pas d'inversion silencieuse de classe** : score moyen > 0,8 sur les parasitées, < 0,2 sur les saines ;
- taille de l'ONNX < 20 Mo ;
- `class_map` de la fiche identique à la config, seuil dans]0, 1[.

4. Couverture de code (E2)

- Mesurée sur `src/`, seuil minimal **70 %** (`fail_under`), lignes manquantes affichées.
- Exclus : les scripts offline dépendant de TensorFlow ou de données réelles (`download`, `prepare`, `train`, `model`, `export`, `evaluate`, `calibrate_threshold`). Ils sont validés par l'exécution du pipeline et par les tests de non-régression du modèle.

5. Test de charge (`tests/load/locustfile.py`, E3)

```
locust -f tests/load/locustfile.py --host http://localhost:8000 \
      --headless -u 20 -r 5 -t 2m --csv reports/load
```

- Utilisateur simulé `PredictUser` : attente 0,5 à 1,5 s, 10 appels `/v1/predict` pour 1 appel `/health`.
- Images : jusqu'à 200 PNG de `LOAD_IMAGES_DIR` (défaut `data/raw/cell_images`), sinon 20 images aléatoires.
- `LOAD_API_KEY=k1,k2,...` : une clé par utilisateur, distribuées en rotation.
- À la fin, `enforce_slo` impose **P95 ≤ 500 ms et zéro échec** sur `/v1/predict`, sinon code de sortie 1.

Conditions de mesure : conteneur limité à 2 vCPU / 2 Go (`docker compose`), et `RATE_LIMIT` relevé ou une clé par utilisateur. Sinon, les 429 font échouer le test : 20 utilisateurs dépassent vite 60 requêtes par minute sur une seule clé.

6. Écrire un nouveau test

- Test d'API : utiliser `client` ou `make_client(cfg, <surcharges>)` ; ne jamais dépendre d'un vrai modèle.
- Image de test : `to_bytes(make_cell(True))`.
- Faux modèle ONNX : `OnnxPredictor(session=...)` avec un objet qui expose `get_inputs`, `get_outputs` et `run` (voir `test_inference.py`).
- Toute modification de `preprocessing.py` doit garder `test_preprocessing_reference_values_are_stable` vert, ou s'accompagner d'un réentraînement et d'une nouvelle version de modèle.

Déploiement et exploitation

1. Environnements Python

Python **3.10 ou 3.11** obligatoire (`requires-python = ">=3.10,<3.12"`) : TensorFlow 2.15 ne supporte pas les versions suivantes.

Fichier	Contenu	Usage
<code>requirements.txt</code>	FastAPI, Uvicorn, python-multipart, Pydantic, ONNX Runtime, numpy < 2, Pillow, PyYAML, slowapi	API / image Docker (sans TensorFlow)
<code>requirements-train.txt</code>	+ TensorFlow 2.15.1, tf2onnx, onnx, protobuf, scikit-learn, matplotlib	Pipeline d'entraînement
<code>requirements-dev.txt</code>	+ pytest, pytest-cov, httpx, Locust	Tests

```
py -3.11 -m venv .venv
.venv\Scripts\activate          # Windows ; Linux/macOS : source .venv/bin/activate
pip install -r requirements-train.txt -r requirements-dev.txt
```

2. Image Docker

Dockerfile — image d'inférence `python:3.11-slim` :

1. Installe `requirements.txt` uniquement (image légère, sans TensorFlow).
2. Copie `config/`, `src/`, `web_app/` et `models/`. Le `.dockerignore` exclut données, tests, notebooks, `models/keras` et `models/checkpoints`.
3. Exécute le service sous un utilisateur non root `palunet` (UID 10001) ; seul `/app/logs` est accessible en écriture.
4. Variables par défaut : `PORT=8000`, `LOG_LEVEL=INFO`, `RATE_LIMIT=60/minute`, `INTRA_OP_THREADS=2`.
5. `HEALTHCHECK` toutes les 30 s sur `/health` (readiness : échoue tant que le modèle n'est pas chargé).
6. Lancement : `uvicorn src.api.main:app --host 0.0.0.0 --port $PORT --workers 1 --no-access-log`. Le journal d'accès Uvicorn est désactivé : il contiendrait les adresses IP.

Le modèle doit être présent avant le build (`models/model_v<v>.onnx` + `.json`), ou monté en volume avec `MODEL_PATH`. Sinon le conteneur démarre mais reste non prêt (503).

docker compose

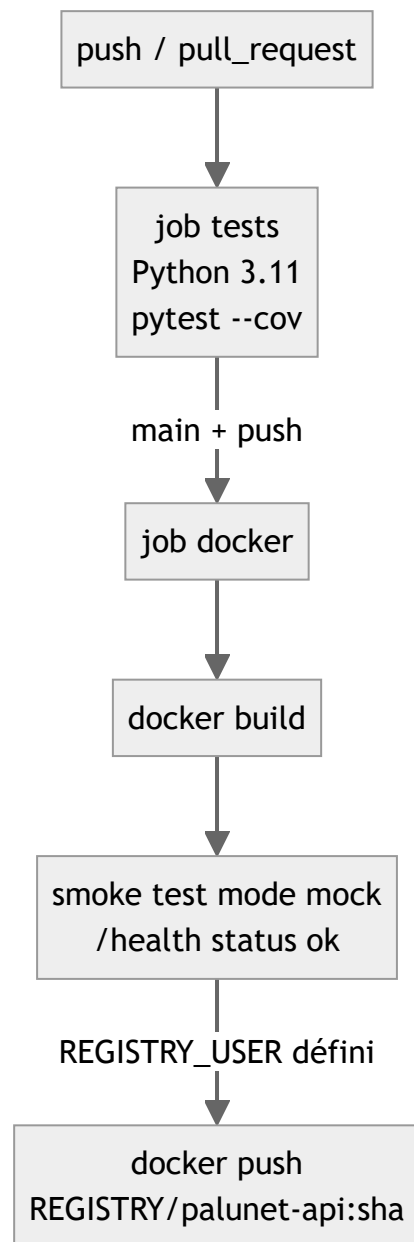
```
cp .env.example .env          # puis définir API_KEY
docker compose up --build
```

`docker-compose.yml` :

- `API_KEY` est **obligatoire** (`${API_KEY:?...}`) : le démarrage échoue sans clé ;
- `MODEL_PATH` par défaut : `models/model_v1.0.0.onnx` (à mettre à jour à chaque version) ;
- journal de prédictions dans `/app/logs/predictions.log`, monté sur `./logs`, 365 jours de rétention ;

- limites de ressources : 2 CPU, 2 Go (E6) ;
- `restart: unless-stopped` .

3. CI/CD — `.github/workflows/ci-cd.yml`



Job	Déclencheur	Étapes
tests	tout push et toute PR	Installation <code>requirements-dev.txt</code> , <code>pytest --cov</code> (seuil 70 %), rapport <code>coverage.xml</code> publié en artefact
docker	push sur <code>main</code> , après tests	Build taggé avec le SHA, démarrage en <code>MOCK_MODEL=true</code> , attente de <code>/health</code> (30 s max), vérification de <code>"status": "ok"</code> , push vers le registre si les secrets existent

Secrets attendus : `REGISTRY` , `REGISTRY_USER` , `REGISTRY_PASSWORD` .

Le modèle n'est pas dans Git : l'image CI est testée en mode mock. Avant un vrai déploiement, il faut récupérer l'artefact ONNX depuis un stockage d'artefacts (non encore automatisé).

4. Mise en production d'une nouvelle version de modèle

1. Exécuter le pipeline complet (voir [README.md](#)).
2. Vérifier les critères d'acceptation dans la sortie de `evaluate` (accuracy $\geq$ 0,95, recall $\geq$ 0,96, AUC $\geq$ 0,97) et le jeu externe.
3. Committer `metrics_baseline/<v>.json`.
4. Publier `model_v<v>.onnx` et `model_v<v>.json` dans le stockage d'artefacts. Contrôler le `sha256` de la fiche.
5. Déployer avec `MODEL_PATH=models/model_v<v>.onnx` ; vérifier `/health` (version, seuil).
6. Garder la version précédente disponible pour un retour arrière immédiat (changer `MODEL_PATH`).

5. Journaux

Tous les logs sont des **lignes JSON** sur stdout (`src/utils/logger.py`) :

Champ	Contenu
<code>timestamp</code>	ISO 8601 UTC
<code>level</code> , <code>logger</code> , <code>message</code>	Standard
champs <code>extra</code>	Ajoutés tels quels (ex. <code>model_version</code> , <code>prediction_id</code>)
<code>exception</code>	Trace si l'enregistrement en porte une

Fonction	Rôle
<code>JsonFormatter</code>	Séréalise l'enregistrement et ses champs <code>extra</code> (hors attributs standard de <code>LogRecord</code>)
<code>configure_logging(level, prediction_log_file, retention_days)</code>	Remplace les handlers racine par un handler JSON sur stdout (force l'UTF-8 pour les consoles Windows). Si <code>prediction_log_file</code> est fourni, ajoute au logger <code>palunet.predictions</code> un <code>TimedRotatingFileHandler</code> (rotation à minuit UTC, <code>retention_days</code> fichiers conservés : purge automatique au-delà de 12 mois)
<code>get_logger(name)</code>	Logger nommé ; configure la journalisation par défaut si rien ne l'a encore fait (scripts CLI)

Le journal des prédictions part **aussi** sur stdout (propagation vers la racine). Les contraintes d'anonymisation sont décrites dans [api.md](#) et [gouvernance-donnees.md](#).

6. Supervision

Sonde	Usage
<code>GET /health/live</code>	Liveness : redémarrer le conteneur si elle échoue
<code>GET /health</code>	Readiness : n'envoyer du trafic que si 200
<code>GET /metrics</code>	Latences p50/p95/p99, taux d'erreur, codes HTTP, version, seuil (par instance)

Alertes conseillées : P95 > 500 ms, `error_rate` en hausse (attention : il inclut les 4xx client), `/health` en 503, changement inattendu de `model_version` ou de `decision_threshold`.

7. Suivi de la dérive (E5)

Procédure détaillée dans plan-derive.md. En résumé :

```
python -m src.models.evaluate --images-dir audit/2026-10
```

Code de sortie 2 si le rappel est inférieur à 94 % : déclencher le réentraînement. Entre 94 % et 96 % : surveillance renforcée.

8. Dépannage

Symptôme	Cause probable	Action
/health 503 « Modèle introuvable »	MODEL_PATH erroné ou modèle non copié dans l'image	Vérifier models/ ou monter le volume
/health 503 « Fiche modèle introuvable »	.json absent à côté du .onnx	Copier la fiche (même nom de base)
/health 503 « Aucun seuil calibré »	Calibration non lancée	python -m src.models.calibrate_threshold ou DECISION_THRESHOLD
/health 503 « class_map du modèle ... != config »	Modèle entraîné avec un autre encodage	Utiliser le bon modèle ou la config correspondante, ne pas forcer
Le conteneur s'arrête au démarrage	Fichier ONNX corrompu (erreur ONNX Runtime, pas ModelLoadError)	Vérifier le sha256 de la fiche
401 sur toutes les requêtes	X-API-Key absent ou différent de API_KEY	Vérifier la clé (plusieurs clés séparées par des virgules)
429 fréquents	Quota par clé atteint ou partagé derrière un proxy (quota par IP)	Relever RATE_LIMIT, une clé par client, Redis pour plusieurs instances
Quota incohérent entre instances	Stockage memory:// par instance	RATE_LIMIT_STORAGE=redis://...
Latence élevée	Sur-souscription CPU	INTRA_OP_THREADS = nombre de vCPU
pip install échoue sur TensorFlow	Python ≥ 3.12	Utiliser Python 3.10 ou 3.11
Caractères illisibles dans la console Windows	Encodage cp1252	Les logs forcent l'UTF-8 ; utiliser Windows Terminal
prepare : « Aucune image trouvée »	Dataset non téléchargé	python -m src.data.download

Points d'attention, limites et pistes d'amélioration

Constats issus de l'analyse du code au 5 octobre 2026 (commit `3c981d0`). Ils ne remettent pas en cause le fonctionnement de la V1 de démonstration, mais doivent être traités ou acceptés avant un usage réel.

Gravité : **Haute** = à corriger avant production ; **Moyenne** = risque d'exploitation ; **Basse** = confort ou maintenabilité.

1. Robustesse et sécurité de l'API

#	Gravité	Constat	Effet	Piste
1	Moyenne	<code>lifespan</code> n'intercepte que <code>ModelLoadError</code> . Une erreur d'ONNX Runtime (fichier corrompu) ou le <code>ValueError</code> d' <code>OnnxPredictor</code> (sortie <code>probability</code> absente) n'est pas convertie	Le processus s'arrête au démarrage au lieu de rester vivant avec <code>/health = 503</code>	Dans <code>from_settings</code> , encapsuler la création d' <code>OnnxPredictor</code> et relever <code>ModelLoadError</code>
2	Moyenne	Le <code>sha256</code> de la fiche modèle n'est pas vérifié au chargement	Un artefact modifié ou incomplet peut être servi sans alerte	Comparer le hash au démarrage (coût : une lecture du fichier)
3	Moyenne	Quand l'authentification est désactivée, la clé de quota est quand même <code>key: <X-API-Key></code> si l'en-tête est présent	Un client contourne le quota par IP en envoyant un en-tête différent à chaque requête	Utiliser la clé API seulement si l'authentification est active (<code>settings.api_keys</code> non vide)
4	Basse	Un lot de 32 images compte pour une seule requête dans le quota	Débit effectif jusqu'à 32 fois le quota annoncé	Quota distinct pour <code>/batch</code> ou coût proportionnel au nombre d'images
5	Basse	<code>/metrics</code> est public et expose version et seuil	Information utile à un attaquant, sans données patient	Restreindre au réseau interne ou derrière la clé API
6	Basse	La clé API de l'interface web est stockée en clair dans <code>localStorage</code>	Lisible par tout script de la page ou par un autre utilisateur du poste	Acceptable en démonstration ; prévoir une authentification par session ensuite

2. Observabilité

#	Gravité	Constat	Piste
7	Basse	<code>error_rate</code> compte tous les codes ≥ 400 (401, 413, 429 inclus)	Exposer séparément les 4xx et les 5xx pour l'alerting
8	Basse	Métriques en mémoire, par processus, remises à zéro au redémarrage, format JSON propriétaire	Export Prometheus si une supervision centralisée est mise en place
9	Basse	Percentiles par la méthode du rang le plus proche, sur les 1 000 dernières requêtes	Suffisant pour une instance ; documenter la fenêtre dans les tableaux de bord

3. Modèle et données

#	Gravité	Constat	Effet	Piste
10	Haute	Le jeu externe (A6, ≥ 200 images de laboratoire) n'est pas encore fourni	Performance hors NIH inconnue ; c'est le principal risque clinique	Bloquant pour la recette (voir gouvernance-donnees.md)
11	Moyenne	Redimensionnement en 128×128 sans conserver le ratio et sans normalisation de couleur	Sensible aux changements de microscope, de grossissement ou de coloration	Surveiller la dérive (E5) ; envisager une normalisation de coloration si les audits le montrent
12	Moyenne	La formule Grad-CAM de <code>build_inference_model</code> suppose exactement la tête GAP $\rightarrow$ Dense ReLU $\rightarrow$ Dense	Toute modification de la tête rend la carte fausse, sans erreur	Ajouter un test comparant la carte analytique à un Grad-CAM par <code>GradientTape</code> sur le modèle Keras
13	Basse	La carte Grad-CAM a une résolution native de 4×4 (agrandie en 128×128)	Localisation grossière : indique une zone, pas un parasite précis	À mentionner aux utilisateurs
14	Basse	<code>train.py</code> charge tout le cache en mémoire (environ 1,4 Go pour le dataset complet)	Échec possible sur une petite machine	<code>--limit</code> pour les essais ; <code>tf.data</code> depuis le disque si nécessaire
15	Basse	En cas de quantification, <code>model_v<v>_fp32.onnx</code> reste dans <code>models/</code> et est copié dans l'image Docker	Image plus lourde	Ajouter <code>models/*_fp32.onnx</code> au <code>.dockerignore</code>

4. Configuration et maintenabilité

#	Gravité	Constat	Piste
16	Basse	<code>load_config()</code> est mis en cache sur l'argument <code>path</code> : <code>CONFIG_PATH</code> n'est lu qu'au premier appel	Documenté ici ; appeler <code>load_config.cache_clear()</code> dans les tests qui changent <code>CONFIG_PATH</code>
17	Basse	<code>model.backbone</code> dans <code>config.yaml</code> n'est lu par aucun code (MobileNetV2 est codé en dur)	Le supprimer, ou l'utiliser pour choisir le backbone
18	Basse	<code>docker-compose.yaml</code> fixe <code>MODEL_PATH</code> par défaut à <code>model_v1.0.0.onnx</code>	À mettre à jour à chaque version, ou dériver de la config
19	Basse	<code>MockPredictor</code> suppose un côté d'image multiple de 4 pour la carte <code>cam</code>	Sans effet avec 128 px ; à adapter si <code>image_size</code> change
20	Basse	La récupération de l'artefact modèle en CI/CD n'est pas automatisée	Brancher un stockage d'artefacts ou un registre de modèles dans le job <code>docker</code>
21	Basse	Les tests de non-régression du modèle sont toujours ignorés en CI (pas de modèle)	Les exécuter dans un job dédié qui télécharge l'artefact et le <code>test.npz</code>

5. Points forts à préserver

- Split **par patient**, testé, qui évite les métriques gonflées.
- Prétraitement **unique** partagé entre entraînement et API, avec tests de valeurs de référence.

- Encodage de classe figé et vérifié au démarrage (B9) ; interprétation centralisée dans `positive_probability`.
- Seuil **calibré sur la validation**, sur l'artefact ONNX réellement servi, jamais 0,5 par défaut.
- API **non prête** plutôt que fausse : absence de modèle, de fiche, de seuil ou incohérence → 503.
- Journalisation anonymisée testée (pas de nom de fichier, pas d'IP).
- Grad-CAM calculé dans le graphe ONNX : pas de TensorFlow en production.